

Complexity Classes

Decision Problems

A Decision Problem is a problem where the answer is either YES or NO. Every optimisation problem has a decision counterpart.

- Optimisation: find the shortest path between u and v in a graph G
- Decision: does there exist a path of length at most k between u and v in G ?

An optimisation problem has a polynomial algorithm iff the corresponding decision problem has a polynomial algorithm.

Every decidable problem has a set of algorithms (or Turing Machines) that solve it. We classify problems based on the efficiency of the most efficient possible algorithm that solves it in terms of Time and Space.

Definition

The **time complexity** of a Turing Machine T is the **function** Time_T such that $\text{Time}_T(x)$ is the **number of steps** taken by the computation $T(x)$

Definition

The **space complexity** of a Turing Machine T is the **function** Space_T such that $\text{Space}_T(x)$ is the **number of distinct tape cells** visited during the computation $T(x)$

Note: if the program $T(x)$ does not halt, both the time complexity and space complexity are undefined.

We can generalise time complexity using Big-O Notation:

Definition

For any function f , we say that the **time complexity** of a decidable language $\mathcal{L}$ is $O(f)$, or $\mathcal{L}$ is decidable in $O(f)$ time, if there exists a TM T which decides $\mathcal{L}$, and constants n_0 and c such that for all inputs x with $|x| > n_0$,

$$\text{Time}_T(x) \leq c \cdot f(|x|)$$

We can classify decidable problems (languages) into classes according to their Big-O time complexity.

The complexity class **TIME**[f] (or **DTIME**[f]) is the class of all problems for which an algorithm exists with time complexity $O(f)$.

The Complexity Class **P**:

The complexity class **P** is defined as the union of classes **TIME**[n^k] for all $k \geq 0$, i.e. the class of all problems with finite-degree polynomial time complexity.

P represents the class of problems which are tractable in practice (but not always - $n^{100000!}$ is not necessarily “tractable” but for naturally defined problems polynomials and constants generally do not become stupidly large).

The class **P** is the same regardless of the model of computation or encoding used to represent the problem (as long as we don't use unary). This means that these algorithmic implementation details are not necessary to prove that a problem is in **P**.

Examples of problems in **P** include *2-SATISFIABILITY* and *2-COLOURABILITY*.

Another way we can show that a problem is in **P** is through polynomial reduction.

Formally, a language L_1 is polynomially reducible to L_2 if a polynomial-time computable function f exists such that for all x , x is in L_1 iff $f(x)$ is in L_2 . We write this as $L_1 \leq L_2$.

If $L_1 \leq L_2$ and L_2 is in **P** then L_1 is also in **P** (since the composition of polynomials is also a polynomial).

Example: $2\text{-COLOURABILITY} \leq 2\text{-SATISFIABILITY}$

2-COLOURABILITY: For a graph G does there exist a colouring using at most 2 colours such that no two adjacent vertices are assigned the same colour.

2-SATISFIABILITY: Given a CNF formula where each clause has at most 2 terms, is it satisfiable?

To reduce 2-COLOURABILITY to 2-SATISFIABILITY:

For each vertex v_i of the graph G , create a variable x_i in our CNF formula.

For each edge (v_i, v_j) in G , add two clauses $(x_i \vee x_j)$ and $(\neg x_i \vee \neg x_j)$.

This reduction process is completable in polynomial time - $O(|V| + |E|)$

Since there are maximum 2 colours in our colouring, we can represent these 2 colours as TRUE and FALSE. The two clauses for each edge are only true if both vertices can be assigned a different colour in the colouring, and we need all clauses in a CNF formula to be true for the formula to be satisfiable. Likewise, if the formula is satisfiable define the colouring by setting vertices corresponding to TRUE variables in the satisfying variable assignment to one colour and FALSE variables to the other colour.

The Complexity Class NP

A Turing Machine V is said to be a verifier for a language L if it halts on all inputs and $L = \{ w \mid V \text{ accepts } (w, c) \text{ for some "certificate" } c \}$

I.e. a verifier returns True or False depending on whether an input w is in L .

Such a verifier is polynomial-time if there is a polynomial $p(x)$ such that for any w in L there is a certificate c with $|c| \leq p(|w|)$.

All problems in **P** have polynomial-time verifiers (obviously - we can solve the whole ass problem in poly time so we can definitely verify it in poly-time).

The class **NP** is the class of problems that have polynomial-time verifiers.

Examples of problems in **NP** include *COMPOSITE NUMBER* and *SUBSET SUM*.

An alternative but equivalent definition for **NP** can be obtained using Non-Deterministic Turing Machines.

Definition

The **time complexity** of a **non-deterministic** Turing Machine NT is the function $NTime_{NT}$ such that $NTime_{NT}(x)$ is the number of steps in the shortest accepting path of $NT(x)$ if there is one, otherwise it is the number of steps in the shortest rejecting path.

Note: if not all paths of $NT(x)$ halt, then the time complexity is undefined.

Definition

For any function f , we say that the **non-deterministic** time complexity of a decidable language $\mathcal{L}$ is $O(f)$ if there exists a non-deterministic TM NT which decides $\mathcal{L}$, and constants n_0 and c such that for all inputs x with $|x| > n_0$,

$$NTime_{NT}(x) \leq c \cdot f(|x|)$$

The complexity class **NTIME** $[f]$ is defined to be the class of problems for which an algorithm exists with non-deterministic time complexity $O(f)$.

NP can be defined as the union of classes **NTIME** $[n^k]$ for all $k \geq 0$.

We do not currently know for sure whether or not **P** = **NP**.

The hardest problems in **NP** are known as **NP-Complete**. These are all equally difficult, and all languages in **NP** can be reduced to any **NP-Complete** language in poly-time.

To prove that a language is **NP-Complete** we have to show that it is in **NP** and that another known **NP-Complete** language is poly-time reducible to it. (Thankfully some nerds already found the first **NP-Complete** language).

Note: If we can reduce another known **NP-Complete** language to a language but not prove it is in **NP**, then it is **NP-Hard**.

Example: Prove *VERTEX COVER* is NP-Complete

VERTEX COVER: given a graph $G = (V, E)$ and a natural number k , is there a subset W of V with $|W| \leq k$, such that for each edge (i, j) of G , $\{i, j\} \cap W \neq \{\}$ - i.e. W contains at least one endpoint of every edge in the graph.

This problem is in **NP** as the list of vertices in the cover is the certificate.

We can prove this is **NP-Complete** by reducing *SATISFIABILITY* to it:

Given a formula f of n variables and clauses $C_1, \dots, C_m$:

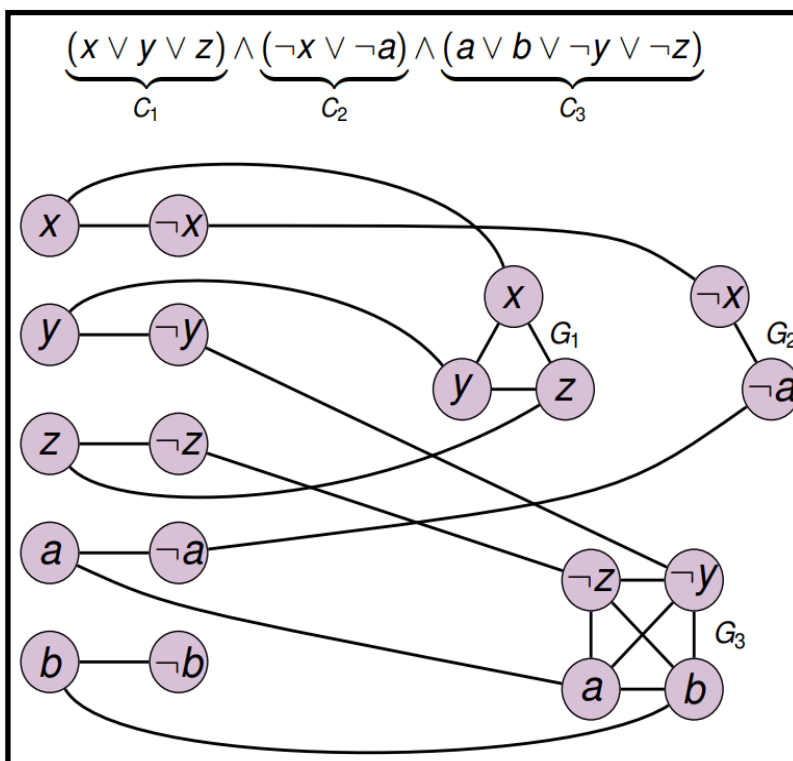
For each variable x , create two adjacent vertices x^t and x^f (with an edge between them).

For each clause C_i , create a new complete subgraph G_i with all possible internal edges, and with an edge from each vertex to its corresponding variable vertex (created $\sum_{j=1}^m$ in the previous step)

Set $k = n + \sum_{j=1}^m (n_j - 1)$, where n_j is the number of vertices in graph G_j .

Note: k is the maximum size of vertexcover we are looking for

Example construction:



For each variable x , at least one of the graph nodes x^f and x^t must be in the vertex cover (to cover the edge between them).

For each clause graph G_i , at least $n_i - 1$ vertices must be included in the cover (to cover all the edges in the complete graph)

If the CNF formula is satisfiable, include each literal assigned TRUE in the cover, and all but one vertex in each G_i .

Conversely, if a vertex cover exists, assign each Boolean variable according whether the vertices corresponding to x^t or x^f are in the cover.

Because of how we defined k (the $n_j - 1$ thing), there must be exactly one vertex in each G_i which is not in the covering. Since the edge from that vertex to its corresponding variable assignment is included in the covering (all edges must be included for a covering to exist), this means that the literal vertex at the other endpoint of this edge is in the cover, and hence at least one literal in each clause is satisfied by the variable assignment.

Therefore every clause is satisfied iff a vertex cover for this graph exists.

=> VERTEX COVER is an **NP-Complete** problem.

Example: Prove that *CLIQUE* is NP-Complete

CLIQUE: Given a finite graph $G = (V, E)$ and an integer k , does G have a clique (subset of vertices with all possible edges between them) of size k ?

CLIQUE is in **NP** since the certificate is the clique of size k , which can be verified in poly-time.

We can reduce *VERTEX COVER* to *CLIQUE* - a set of vertices W is a vertex cover in G iff $V - W$ is a clique in the complement of G .

Note: the complement of a graph is a graph with the same vertex set but where two vertices are connected by an edge iff they weren't in the original graph.

=> *CLIQUE* is **NP-Complete**.

Example: Prove that *3-SATISFIABILITY* is NP-Complete

3-SATISFIABILITY is the satisfiability problem where all clauses are of size 3 (or less). This problem is in **NP** since we can easily verify whether a given variable assignment is valid.

We can reduce *SATISFIABILITY* to *3-SATISFIABILITY* by replacing every clause C with $k > 3$ as follows:

$$C = x_1 \vee x_2 \vee \dots \vee x_k$$

$$C' = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee y_3) \wedge \dots \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

Then, C is satisfiable iff C' is satisfiable. This is because for C to be satisfied, at least one of $x_1, \dots, x_k$ must be True, and by the construction of C' , if all 'x's are False, C' cannot be satisfied but if at least one is True, there exists some assignment of the 'y's such that C' is satisfied.

(e.g. if x_3 is True, set $y_1 = \text{True}$ and $y_2, \dots, y_{k-3} = \text{False}$ to satisfy C')

=> *3-SATISFIABILITY* is **NP-Complete**.

Example: Reduce *HAMILTONIAN CYCLE* to *TRAVELLING SALESMAN*

HAMILTONIAN CYCLE: given a graph $G = (V, E)$ is there a cycle in G that visits

each vertex exactly once?

TRAVELLING SALESMAN: Given a finite set of cities $\{c_1, \dots, c_n\}$, a positive integer distance $d(i, j)$ between each pair of cities (c_i, c_j) and an integer B , is there a permutation that visits every city once and returns to the start such that the total distance of the tour is at most B ?

TRAVELLING SALESMAN is in **NP** since a route permutation is the certificate, the total distance of this route can be verified in poly-time by summing the distances between adjacent cities in the permutation.

We can reduce *HAMILTONIAN CYCLE* to *TRAVELLING SALESMAN* as follows:

Given a graph $G = (V, E)$:

- For each vertex v create a city c_v in the TSP tour
- For each pair of distinct vertices (u, v) , set $d(c_u, c_v) = 1$ in the tour if the edge (u, v) exists in G , otherwise set $d(c_u, c_v) = 2$.
- Set $B = |V|$, i.e. the number of vertices in G .

Then, if G has a Hamiltonian cycle, this cycle is a route in the TSP tour of cost exactly B . If there exists a route of cost B in the TSP tour then it must not have used pairs with distance 2, and so edges of this tour must correspond to edges in G . Since we go back to the start in a TSP tour, this must correspond to a Hamiltonian cycle in G .

$\Rightarrow$ *TRAVELLING SALESMAN* is **NP-Complete**.

Strong vs Weak NP-Completeness

NP-Complete problems which remain NP-Complete when all input numbers are bounded by some polynomial in the length of the input are called **Strongly NP-Complete**. This includes all NP-Complete problems without numerical data, e.g. *SATISFIABILITY* and graph problems.

$\rightarrow$ *TRAVELLING SALESMAN* is **Strongly NP-Complete** as it is NP-Complete even if all distances are 1 and 2 (as in the previous reduction example).

NP-Complete problems that are not Strongly NP-Complete are called **Weakly**

NP-Complete.

Example: (0-1) KNAPSACK

KNAPSACK: Given a set of items $I = \{i_1, \dots, i_n\}$ where each i_j has value v_j and weight w_j , and a target value t and a weight upper limit L . Does there exist a subset T of I such that the total value is at least t and the total weight is at most L ?

KNAPSACK is in **NP** since the subset T is the certificate, which can easily be verified by summing the values and weights.

We can reduce a known NP-Complete problem **SUBSET SUM** to **KNAPSACK** to prove NP-Completeness:

SUBSET SUM: given a set of positive integers $S = \{n_1, \dots, n_k\}$ and a target integer r , does there exist some subset T of S such that the sum of all integers in the subset is equal to r ?

Consider instances of **KNAPSACK** such that $v_j = w_j (= n_j$ in the Subset Sum) for all j , and set $t = L (= r$ in the Subset Sum). Then we have that the sum of the values is at least t and the sum of the weights is at most L if and only if the sum of the " n_i "s in the subset sum is equal to r .

$\Rightarrow$ **KNAPSACK** is NP-Complete.

But: **KNAPSACK** can be solved in $O(nL)$ time using dynamic programming, so shouldn't it be in **P**? The dynamic programming solution is linear in the value of L , but time complexity is measured in terms of the length of the input (in binary, or any non-unary encoding), not its value. Hence, $O(nL)$ is actually exponential, and thus is **Weakly NP-Complete** - if we bounded L by some polynomial of n , then these instances of **KNAPSACK** would be solvable in poly-time.

The Complexity Class Co-NP

The class **Co-NP** is the class of languages L such that the complement of L has a polynomial time verifier.

$\rightarrow$ **NP** is the class of problems whose yes-instances can be verified in poly-time; **Co-NP** is the class of problems whose no-instances can be verified in poly-time.

Examples of problems in **Co-NP** include *TAUTOLOGY* and *NO HAMILTONIAN CYCLE*.

TAUTOLOGY: Given a logical formula f , is f satisfied by all possible variable assignments?

-> Complement of *TAUTOLOGY*: Does there exist some truth assignment that unsatisfies f ? I.e. does there exist some truth assignment for $\neg f$? This is equivalent to *SATISFIABILITY*, which is in **NP**. Therefore the no-instances of *TAUTOLOGY* can be verified in poly-time - the certificate for the no-instances is the truth assignment that un-satisfies f -> *TAUTOLOGY* is in **Co-NP**.

NO HAMILTONIAN CYCLE - given a graph, is it true that it has no Hamiltonian cycle?

-> Complement of this is *HAMILTONIAN CYCLE* which is in **NP**. A Hamiltonian cycle is a certificate for no-instances which can be verified in poly-time (since we know *HAMILTONIAN CYCLE* is in NP). Therefore, *NO HAMILTONIAN CYCLE* is in **Co-NP**.

The hardest problems in **Co-NP** are called **Co-NP-Complete**. As with NP-Completeness, all problems in **Co-NP** can be polynomially reduced to any **Co-NP-Complete** problem.

Note: A language is **Co-NP-Complete** if and only if its complement is **NP-Complete**.

=> this means that *TAUTOLOGY* and *NO HAMILTONIAN CYCLE* are both **Co-NP-Complete**.

We do not currently know whether **Co-NP = NP** (as with **P = NP**), but it is generally thought that **P**, **NP** and **Co-NP** are all different. All problems in **P** are also in **NP** and **Co-NP** as both yes and no instances of **P** problems can be verified in poly-time.

Optimisation Problems

Optimisation problems aim to find some maximum or minimum value. In the

real world, the optimisation version of a problem is often more useful than the decision version.

Example:

VERTEX COVER (decision): Given a graph G and an integer k , does G have a vertex cover of size at most k ?

VERTEX COVER (optimisation): Given a graph G , what is the smallest vertex cover of G ?

The optimisation version of a decision problem is at least as hard as the decision version - the algorithm for an optimisation would naturally provide a solution to the decision problem. Therefore, the optimisation version of an **NP-Complete** decision problem is **NP-Hard**.

Note: **NP-Hard** = *at least as hard as NP-Complete*. Also includes problems in a class outside **NP**, i.e. those that cannot be verified in poly-time.

Optimisation problems that are **NP-Hard** are said to be **intractable**.

If we want to solve such a problem, we can:

- Look at *special cases* of the problem which are tractable
- Use *heuristics* - algorithms which generally produce a “good” solution but where no guarantee of performance is proven
- Use *approximation algorithms* which guarantee being within a certain factor of the optimal solution.

An algorithm is a **k -approximation** of an optimisation problem if it always finds a solution that is within a factor of k from the optimum.

- For minimisation problems, this is when it finds a solution that is at least k times the optimum
- For maximisation problems, this is when it finds a solution where the optimum is at most k times as big.

Example: 2-approximation of *VERTEX COVER*:

APPROX-VERTEX-COVER($G = (V, E)$)

1 $C \leftarrow \emptyset$

2 $E' \leftarrow E(G)$

3 **while** $E' \neq \emptyset$ **do**

4 let (u, v) be an arbitrary edge of E'

5 $C \leftarrow C \cup \{u, v\}$

6 remove from E' every edge incident with either u or v

7 **return** C

If we had a Path $a - b - c - d$ as our graph G , then if the algorithm chose the edge (b, c) first, it would add b and c to the vertex cover and remove both other edges, producing an optimal vertex cover of size 2, BUT if it chose the edge (a, b) first, it would add a and b to the vertex cover, removing (b, c) , and then it will have to pick the only remaining edge (c, d) , where c and d are added to the vertex cover. This produces a vertex cover of size 4, which is exactly 2 times the optimal.